

Problema AB23. TRIANGLE

 10 sec.  1024 MB

Noua conducere a comitetului național, reprezentată de coordonatorii grupurilor A și B, dorește să mențină o programă strictă a problemelor date pentru selectarea echipei naționale. În acest scop, ei vor avea nevoie să dea o problemă de geometrie. În plus – numărul de probleme interactive date până acum este sub standard. Pentru a rezolva criza în alegerea problemelor pentru IOI, Sashka a decis să dea o problemă care este și interactivă, și geometrică. Ea are următorul enunț:

Juriul a ascuns o permutare $P_0, P_1, \dots, P_{N-1}$ a $\{1, 2, \dots, N\}$. Trebuie să găsiți permutarea. În acest scop, puteți întreba juriul următoarea întrebare: “Este posibil să se formeze un triunghi cu arie pozitivă cu laturile P_A, P_B, P_C ?”

Se cunoaște (conform inegalității triunghiului) că acest lucru este posibil dacă și numai dacă:

$$P_A + P_B > P_C, P_A + P_C > P_B \text{ și } P_B + P_C > P_A.$$

Scrieți un program **triangle**, care conține funcția **solve**, care va fi compilată împreună cu programul juriului și care va comunica cu acesta, punându-i întrebări de tipul descris mai sus. La finalul execuției, programul trebuie să determine permutarea.

Detalii de implementare

Trebuie să implementați funcția **solve**:

```
std::vector<int> solve(int N)
```

- N : lungimea permutării.

Această funcție va fi apelată de T ori pentru fiecare test – o dată pentru fiecare subtest, fiecare cu N egal, și ar trebui să returneze permutarea ascunsă pentru subtest. Pentru a face asta, programul vostru poate să apeleze funcția juriului **query**:

```
bool query(int A, int B, int C)
```

- A, B, C : indicii laturilor P_A, P_B, P_C .

Funcția returnează **true**, dacă un triunghi cu arie pozitivă și cu laturile P_A, P_B, P_C există și **false**, în cazul contrar.

Restricții

- $N = T = 1\,000$

Subtaskuri

Subtask	Puncte	Descriere
1	100	Subtaskul conține un test cu $T = 1\,000$ de permutări alese aleatoriu și uniform din setul tuturor permutărilor posibile, fiecare cu $N = 1\,000$ de elemente.

Punctele pentru un anumit subtask se obțin numai dacă toate testele acestuia sunt trecute cu succes.

Punctare

Fie $Q_{\text{concurrent}}$ numărul mediu de întrebări pe care programul vostru le face pentru un apel către `solve` pe unicul test, și fie $Q_{\text{țintă}} = 8770$. Atunci scorul vostru pe problemă va fi:

$$\begin{cases} 0 & \text{dacă } Q_{\text{concurrent}} > 2 \times 10^6, \\ 100 & \text{dacă } Q_{\text{concurrent}} \leq Q_{\text{țintă}}, \\ 100 \times \max \left(0.15, 1 - \sqrt{1 - \left(\frac{Q_{\text{țintă}}}{Q_{\text{concurrent}}} \right)^{0.55}} \right) & \text{if } Q_{\text{țintă}} > Q_{\text{țintă}}. \end{cases}$$

Exemplu de interacțiune

Pentru acest exemplu de interacțiune $N = 4$, $T = 2$.

Acțiunea concurentului	Acțiunea juriului
	<code>solve(4)</code>
<code>query(0, 0, 0)</code>	<code>true</code>
<code>query(0, 1, 2)</code>	<code>false</code>
<code>query(0, 1, 3)</code>	<code>false</code>
<code>query(0, 2, 3)</code>	<code>true</code>
<code>query(1, 2, 3)</code>	<code>false</code>
<code>return {3, 1, 2, 4};</code>	
	<code>solve(4)</code>
<code>query(0, 1, 2)</code>	<code>false</code>
<code>query(0, 1, 3)</code>	<code>true</code>
<code>query(0, 2, 3)</code>	<code>false</code>
<code>query(1, 2, 3)</code>	<code>false</code>
<code>return {4, 2, 1, 3};</code>	

Testare locală

Format de intrare:

- linia 1: trei numere întregi T , N , and R - numărul de subteste, mărimea permutărilor, și modul de execuție. Dacă $R = 1$, evaluatorul local va genera uniform aleatoriu permutări și va aștepta un număr pe a doua linie - S , care va fi seed-ul pentru generatorul său de numere aleatorii. Dacă $R = 2$, intrarea va conține:
- liniile de la 2 până la $(1 + T)$: permutări ale $\{1, 2, \dots, N\}$.

Format de ieșire:

- linia 1: un mesaj de eroare sau numărul mediu de întrebări pentru subteste, dacă toate permutările au fost identificate corect.